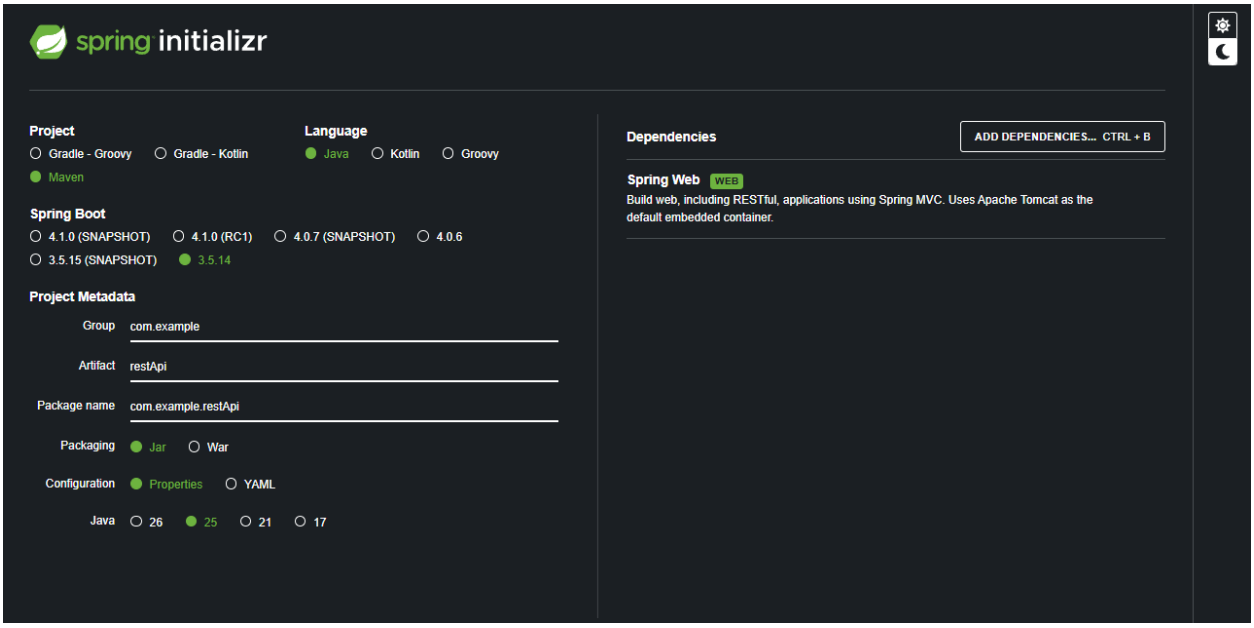
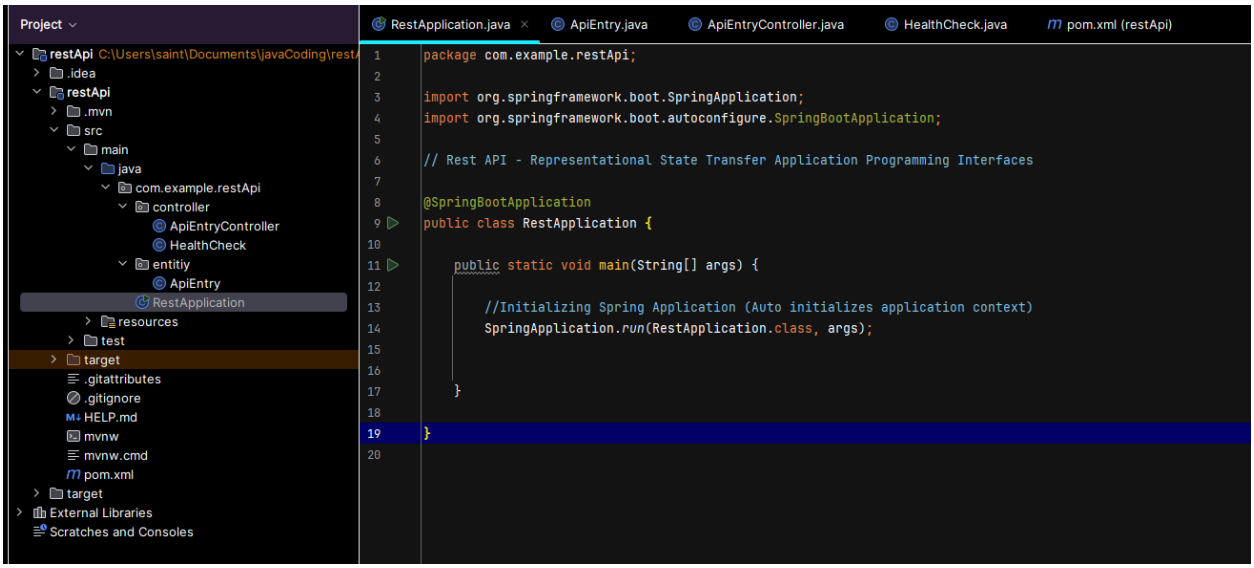
CREATING A BASIC REST API IN SPRING BOOT

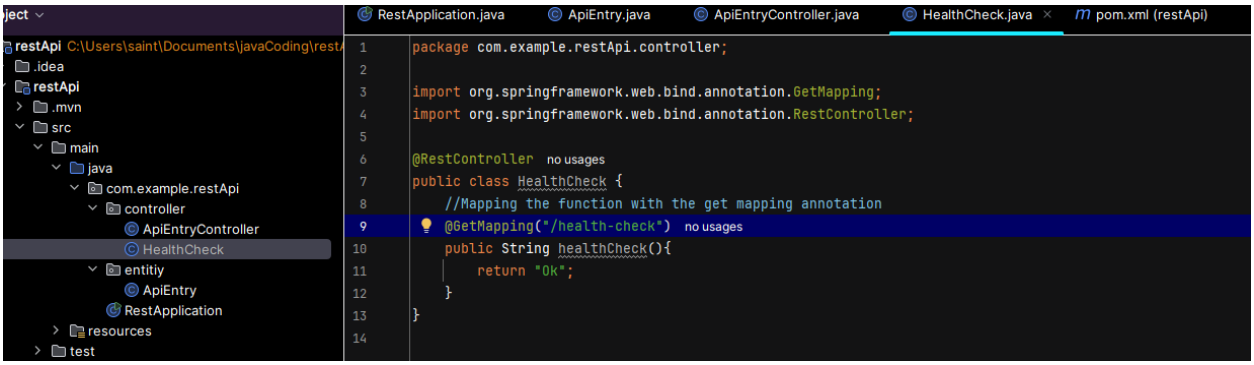
➔ Build a Project in spring initializr.



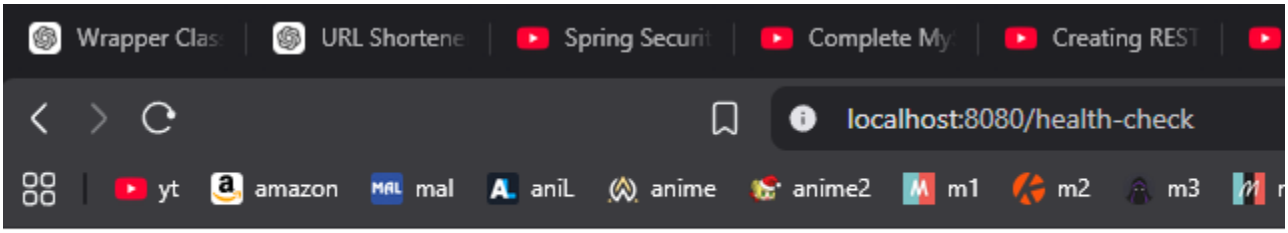
➔ In the main file, adding the @SpringBootApplication annotation and calling the run method.



➔ Create a Controller package and create a basic Health Check Controller with a get mapping.



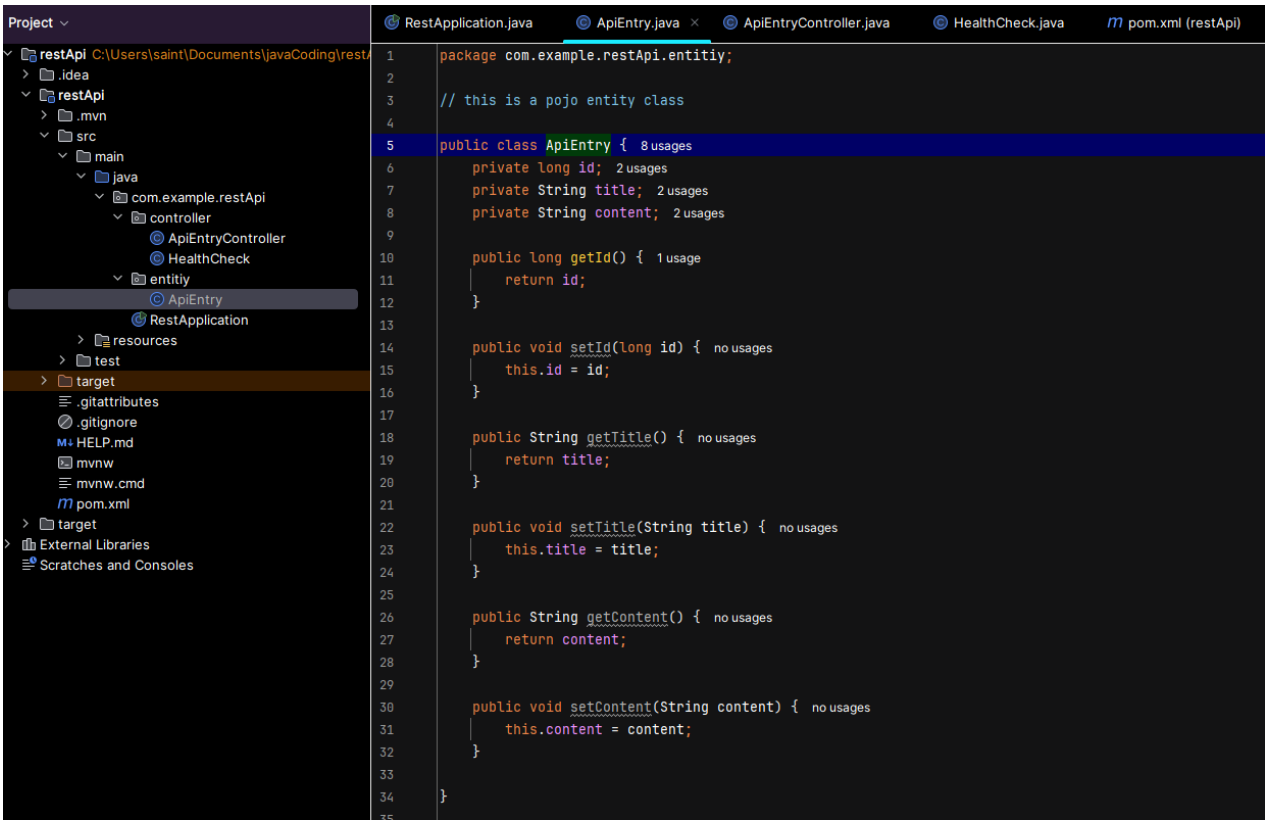
➔ Now if we hit localhost:8080/health-check we can see our html page.



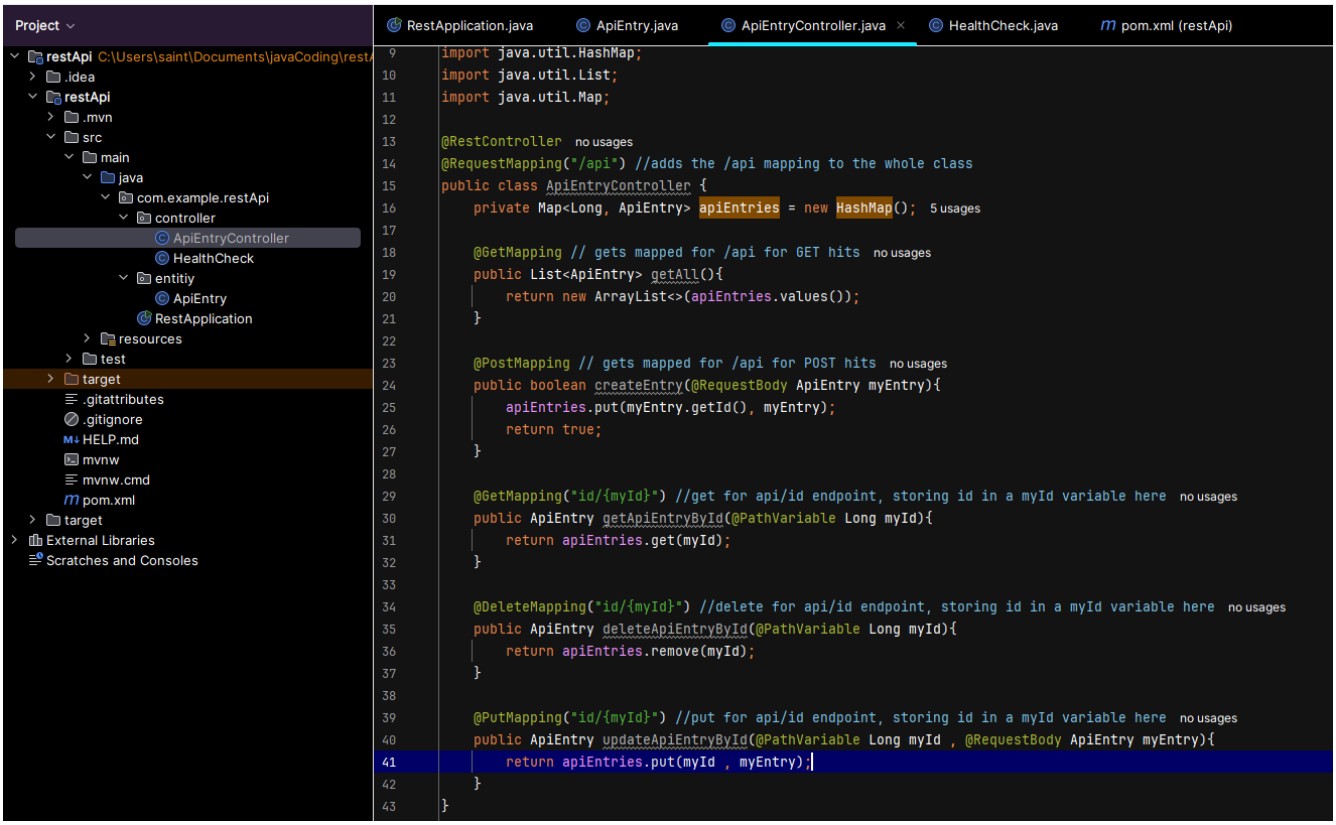
Ok

➔ If hitting on postman, choose raw in Body and choose json.

➔ Create a new package Entity that has a pojo class that defines an entry entity with getters and setters.

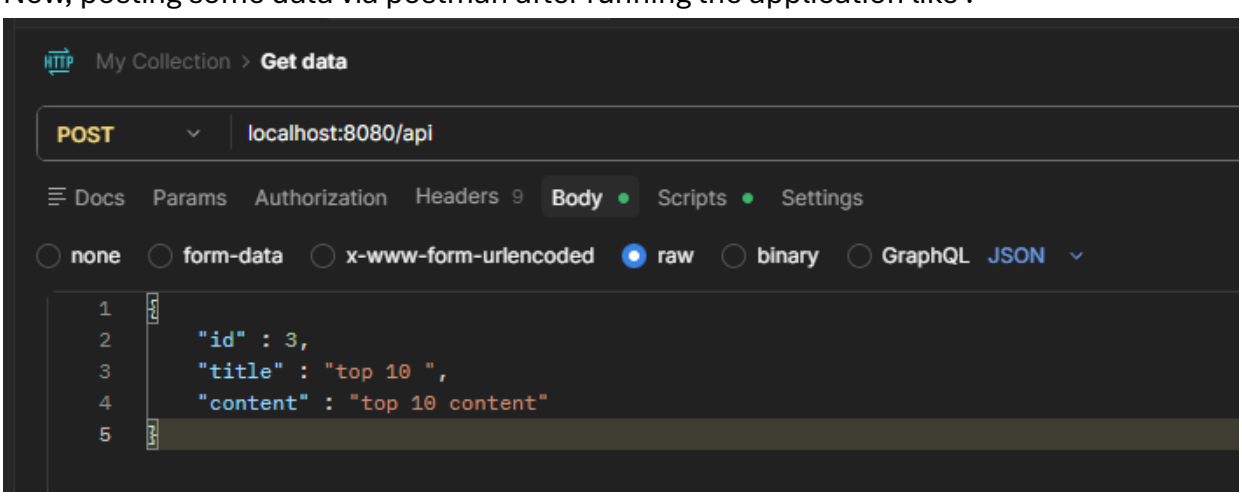


➔ Now create a new Controller ApiEntryController that has all the endpoint request mappings.



- For the example currently we just use a Map that stores the entries and their ids instead of an actual persistent database.
- We use @RequestBody to get the json input from Post and Put calls. Its like saying “Hey spring, [;ease take the data from the request and turn it into a java object that I can use in my code.
- We use @PathVariable to get specific information from the link to delete/put/get for specific ids.
- Methods inside a controller class should be public so that they can be accessed and invoked by the spring framework or external http requests.
- Selecting raw and json in the body of a post request in postman indicates that the request body will contain data in json format, allowing the server to parse and process the incoming data accurately. This ensures that the data is transmitted and received in a structured manner, following the JSON format conventions.

➔ Now, posting some data via postman after running the application like :



➔ Now doing the general GET call on /api gives :

Body Cookies Headers 5 Test Results 1/1

{ } JSON Preview Visualize

```
[
    {
        "id": 1,
        "title": "bad",
        "content": "bad content"
    },
    {
        "id": 2,
        "title": "amazin",
        "content": "amazin content"
    },
    {
        "id": 3,
        "title": "top 444 ",
        "content": "top 4444 content"
    }
]
```

➔ Now doing put calls / delete calls / targeted get calls on a specific id.

PUT localhost:8080/api/id/3

Docs Params Authorization Headers 9 Body Scripts

none form-data x-www-form-urlencoded raw

```
{
    "id": 3,
    "title": "top 444 ",
    "content": "top 4444 content"
}
```